

Actividad: Patrones de creación

Sistema para procesar informes

Sergio Andrés Majé Franco

Reutilización de Software

Universidad Internacional de La Rioja (UNIR)

18 de mayo de 2026

Índice

1. Introducción	2
2. Ejercicio 1: análisis del patrón de creación	2
2.1. Problemas de una implementación sin patrón	2
2.2. Justificación del uso de Abstract Factory	2
2.3. Ventajas e inconvenientes	3
2.4. Por qué no otros patrones de creación	4
3. Ejercicio 2: diseño e implementación en Java	4
3.1. Diseño UML inicial	4
3.2. Estructura principal del código	5
3.3. Resultado de la simulación	9
4. Ejercicio 3: evolución y análisis	9
4.1. Nueva categoría MIXTA	9
4.2. Nuevo tipo PEDIDO en ENTRADA	10
4.3. Valoración de la evolución	10
5. Conclusiones	11

1. Introducción

La actividad plantea el diseño de un sistema capaz de procesar informes a partir del nombre del fichero, siguiendo el formato `CAT_TIPO_nombre.txt`. El dominio inicial contiene dos categorías, ENTRADA y SALIDA, y dos tipos, FACTURA y COMPRA. Además, el sistema debe poder evolucionar con cambios mínimos para soportar una nueva categoría MIXTA y un nuevo tipo PEDIDO solo para la categoría ENTRADA.

El objetivo de la solución no es únicamente que el programa funcione, sino demostrar que el patrón de creación elegido mejora la mantenibilidad, reduce el acoplamiento y permite justificar la evolución del sistema con un coste razonable. Por este motivo se ha empleado una variante pragmática del patrón *Abstract Factory* [2].

2. Ejercicio 1: análisis del patrón de creación

2.1. Problemas de una implementación sin patrón

Una solución directa podría resolver el problema con condicionales anidados:

- analizar el prefijo de cada fichero;
- decidir la categoría mediante `if` o `switch`;
- volver a evaluar el tipo con más condicionales;
- ejecutar en cada rama la lógica concreta de procesado.

Este enfoque tiene varios inconvenientes:

- mezcla análisis sintáctico, selección de estrategia y lógica de negocio en un único punto;
- aumenta el acoplamiento entre el cliente y todas las combinaciones categoría-tipo;
- convierte cada ampliación en una modificación de código ya estable, elevando el riesgo de regresiones;
- dificulta las pruebas unitarias porque la creación de objetos y el flujo de negocio están entrelazados.

2.2. Justificación del uso de Abstract Factory

El patrón *Abstract Factory* permite modelar cada categoría como una familia de productos relacionados [2]. En este caso:

- cada familia representa una categoría de informes;
- cada producto concreto representa el procesador de una combinación categoría-tipo;
- el servicio de aplicación delega la creación en una factoría, en lugar de conocer todas las clases concretas.

La solución implementada utiliza una interfaz de factoría con un método genérico de creación por tipo. Esta decisión mantiene la idea central del patrón, pero evita una versión excesivamente rígida con un método distinto por producto. Ese ajuste es especialmente útil cuando la evolución del sistema no es simétrica, como ocurre al añadir PEDIDO solo a ENTRADA.

2.3. Ventajas e inconvenientes

Ventajas

- desacopla al cliente de las clases concretas;
- agrupa la lógica de creación por familias coherentes;
- mejora la extensibilidad al añadir nuevas categorías;
- facilita las pruebas y la sustitución de implementaciones;
- hace explícita la relación entre una categoría y los tipos válidos para ella.

Inconvenientes

- incrementa el número de clases frente a una solución sencilla;
- añade una capa de indirección que puede parecer excesiva en sistemas muy pequeños;
- en su forma más ortodoxa, añadir nuevos productos obliga a tocar la interfaz abstracta y todas las factorías concretas.

Precisamente por ese último inconveniente, la implementación evita una interfaz cerrada del tipo `createInvoice()` o `createPurchase()` y utiliza un registro por tipo dentro de cada factoría concreta.

2.4. Por qué no otros patrones de creación

Patrón	Ventaja potencial	Motivo para no elegirlo como principal
Factory Method	Simplifica la creación de un producto concreto	No modela tan bien familias de productos relacionadas por categoría.
Builder	Útil cuando el objeto tiene muchos pasos de construcción	Aquí no existe construcción compleja, sino selección entre variantes.
Singleton	Centraliza una instancia global	No resuelve la variabilidad del dominio ni la creación de combinaciones categoría-tipo.

Cuadro 1: Comparación resumida con otros patrones de creación estudiados.

3. Ejercicio 2: diseño e implementación en Java

3.1. Diseño UML inicial

La Figura 1 muestra la versión base del sistema. El flujo es el siguiente:

1. ReportProcessingService recibe los nombres de fichero.
2. ReportParser transforma cada nombre en un objeto ReportMetadata.
3. ReportFactoryProvider resuelve la factoría de la categoría.
4. la factoría concreta crea el ReportProcessor adecuado.

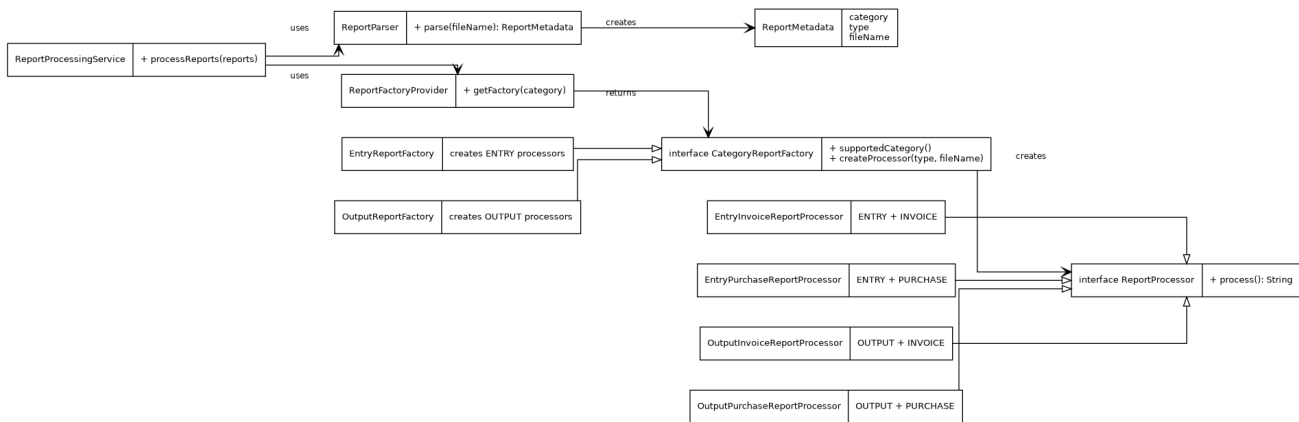


Figura 1: Diseño base del sistema con *Abstract Factory*.

3.2. Estructura principal del código

La interfaz abstracta de factoría mantiene una responsabilidad muy clara: crear procesadores para una familia concreta.

Listing 1: Interfaz de la factoría abstracta.

```

1 package com.unir.reuse.reports;
2
3 /**
4  * Abstract factory contract. Each concrete implementation represents a
5  * category-specific family of processors and decides which report types
6  * are
7  * valid for that family.
8  */
9
10 public interface CategoryReportFactory {
11
12     ReportCategory supportedCategory();
13
14     ReportProcessor createProcessor(ReportType type, String fileName);
15 }

```

Sobre esa interfaz se apoya una implementación base con registro interno. Este punto es relevante porque permite que la familia ENTRADA soporte PEDIDO sin obligar a cambiar el contrato público.

Listing 2: Factoría abstracta con registro por tipo.

```

1 package com.unir.reuse.reports;
2

```

```

3 import java.util.EnumMap;
4 import java.util.Map;
5 import java.util.function.Function;
6
7 /**
8  * Base implementation for concrete factories. A type-to-constructor
9  * registry
10 * keeps the public factory interface stable while still allowing each
11 * category
12 * to support a different subset of report types.
13 */
14 public abstract class AbstractMapBackedReportFactory implements
15     CategoryReportFactory {
16
17     private final Map<ReportType, Function<String, ReportProcessor>>
18         builders =
19         new EnumMap<>(ReportType.class);
20
21     protected final void register(
22         final ReportType type,
23         final Function<String, ReportProcessor> builder
24     ) {
25         builders.put(type, builder);
26     }
27
28     @Override
29     public final ReportProcessor createProcessor(final ReportType type,
30         final String fileName) {
31         Function<String, ReportProcessor> builder = builders.get(type);
32         if (builder == null) {
33             throw new IllegalArgumentException(
34                 "The category %s does not support reports of type %s."
35                     .formatted(supportedCategory().displayName(), type.
36                         displayName())
37             );
38         }
39         return builder.apply(fileName);
40     }
41 }

```

El servicio de aplicación queda desacoplado de las clases concretas y solo orquesta el flujo principal:

Listing 3: Servicio de aplicación.

```
1 package com.unir.reuse.reports;
2
3 import java.util.ArrayList;
4 import java.util.Arrays;
5 import java.util.List;
6
7 /**
8  * Application service that coordinates parsing, factory selection and
9  * processor
10  * execution. The goal is to keep the high-level workflow independent
11  * from the
12  * concrete families and products.
13  */
14 public final class ReportProcessingService {
15
16     private final ReportParser parser;
17     private final ReportFactoryProvider factoryProvider;
18
19     public ReportProcessingService() {
20         this(new ReportParser(), new ReportFactoryProvider());
21     }
22
23     public ReportProcessingService(
24         final ReportParser parser,
25         final ReportFactoryProvider factoryProvider
26     ) {
27         this.parser = parser;
28         this.factoryProvider = factoryProvider;
29     }
30
31     public List<String> processReports(final String[] reportFileNames) {
32         return processReports(Arrays.asList(reportFileNames));
33     }
34
35     public List<String> processReports(final List<String>
36         reportFileNames) {
```

```

34     List<String> results = new ArrayList<>();
35     for (String reportFileName : reportFileNames) {
36         try {
37             ReportMetadata metadata = parser.parse(reportFileName);
38             CategoryReportFactory factory = factoryProvider.
39                 getFactory(metadata.category());
40             ReportProcessor processor =
41                 factory.createProcessor(metadata.type(), metadata.
42                     fileName());
43             results.add(processor.process());
44         } catch (IllegalArgumentException exception) {
45             results.add("Error procesando '%s': %s".formatted(
46                 reportFileName, exception.getMessage()));
47         }
48     }
49     return results;
50 }

```

La ejecución de prueba obligatoria se implementa mediante un main explícito:

Listing 4: Punto de entrada y prueba de funcionamiento.

```

1 package com.unir.reuse.reports;
2
3 /**
4  * Demonstrates the base system and the two requested evolutions with a
5  * single
6  * executable entry point. The last sample intentionally shows a
7  * validation
8  * error to prove that unsupported combinations are rejected explicitly.
9  */
10 public final class Main {
11
12     private Main() {
13     }
14
15     public static void main(final String[] args) {
16         String[] reports = {
17             "ENT_FAC_001.txt",
18             "ENT_COM_002.txt",

```



```

17         "SAL_FAC_003.txt",
18         "SAL_COM_004.txt",
19         "MIX_FAC_005.txt",
20         "MIX_COM_006.txt",
21         "ENT_PED_007.txt",
22         "SAL_PED_008.txt"
23     };
24
25     ReportProcessingService service = new ReportProcessingService();
26     service.processReports(reports).forEach(System.out::println);
27 }
28 }

```

3.3. Resultado de la simulación

La salida por consola verifica tanto los casos válidos como una combinación no soportada:

Listing 5: Salida generada por la ejecución del programa Java.

```

Procesando informe 'ENT_FAC_001.txt' de ENTRADA y tipo FACTURA. Accion simulada: v
Procesando informe 'ENT_COM_002.txt' de ENTRADA y tipo COMPRA. Accion simulada: v
Procesando informe 'SAL_FAC_003.txt' de SALIDA y tipo FACTURA. Accion simulada: en
Procesando informe 'SAL_COM_004.txt' de SALIDA y tipo COMPRA. Accion simulada: ci
Procesando informe 'MIX_FAC_005.txt' de MIXTA y tipo FACTURA. Accion simulada: or
Procesando informe 'MIX_COM_006.txt' de MIXTA y tipo COMPRA. Accion simulada: sin
Procesando informe 'ENT_PED_007.txt' de ENTRADA y tipo PEDIDO. Accion simulada: a
Error procesando 'SAL_PED_008.txt': The category SALIDA does not support reports o

```

4. Ejercicio 3: evolución y análisis

4.1. Nueva categoría MIXTA

La primera evolución incorpora la familia MIXTA. El impacto estructural es limitado:

- se añade `MixedReportFactory`;
- se incorporan dos productos concretos: `MixedInvoiceReportProcessor` y `MixedPurchaseReportProcessor`
- el cliente no cambia, porque sigue trabajando con la abstracción.

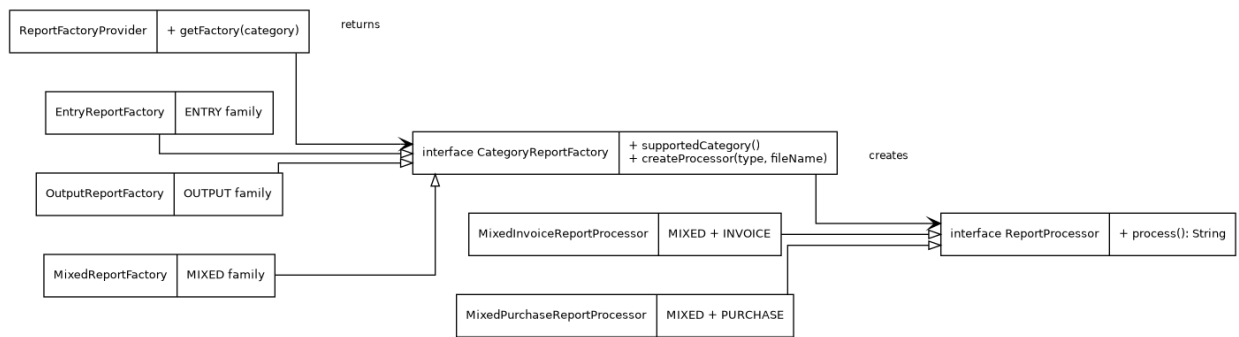


Figura 2: Ampliación del diseño al añadir la categoría MIXTA.

4.2. Nuevo tipo PEDIDO en ENTRADA

La segunda evolución es más interesante desde el punto de vista del patrón, porque el nuevo tipo no aparece en todas las categorías. En una implementación estricta de *Abstract Factory*, esto forzaría cambios en la interfaz abstracta y en todas las factorías concretas. En la solución adoptada:

- se añade el nuevo valor ORDER en la enumeración de tipos;
- se crea EntryOrderReportProcessor;
- solo EntryReportFactory registra ese nuevo producto;
- el resto de familias permanece intacto.

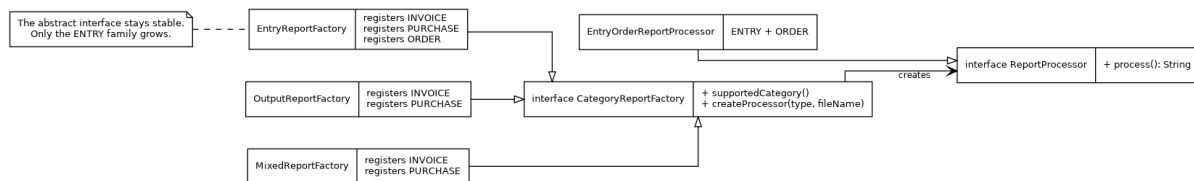


Figura 3: Ampliación del diseño al añadir el tipo PEDIDO en ENTRADA.

4.3. Valoración de la evolución

La solución confirma dos ideas relevantes. Primero, el patrón ayuda claramente cuando se añaden nuevas familias completas, como MIXTA. Segundo, cuando la evolución introduce un producto asimétrico, la implementación concreta del patrón importa tanto como el patrón en sí. La variante basada en registro conserva las ventajas de *Abstract Factory* y evita un coste de cambio innecesario, lo que mejora la reutilización y la mantenibilidad [1, 3].

5. Conclusiones

El uso de *Abstract Factory* resulta adecuado para este problema porque el sistema procesa combinaciones de productos relacionados y, además, el enunciado exige justificar su evolución con cambios mínimos. Frente a una implementación sin patrón, la solución propuesta reduce acoplamiento, mejora la organización del código y hace más evidente dónde deben incorporarse las nuevas variantes del dominio.

La práctica también muestra una lección importante: aplicar un patrón no debe significar copiar una plantilla rígida. En este caso, una adaptación razonada de *Abstract Factory* ofrece un mejor equilibrio entre pureza teórica y coste real de mantenimiento.

Finalmente, el proyecto completo se encuentra disponible en el repositorio <https://github.com/smaje99/software-reuse-unir>. Asimismo, junto a este documento se adjunta un archivo comprimido en formato ZIP que incluye el ejercicio desarrollado en Python, C#, y Java, para facilitar su revisión y ejecución.

Referencias

- [1] Joshua Bloch. *Effective Java*. Addison-Wesley, 3 edition, 2018.
- [2] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [3] Robert C. Martin. *Clean Architecture*. Prentice Hall, 2018.